

Marcar como hecha

Node.js en windows

Aunque es reciente, y ha tardado en llegar mucho más que en otros OS, en windows también existe el nvm (node version manage), que nos permite teniendo instaladas más de una versión, ir cambiado de una a otra con un simple comando.

existe un instalador en:

<https://github.com/coreybutler/nvm-windows>

que lleva a la página de descarga nvm-setup.exe

<https://github.com/coreybutler/nvm-windows/releases>

Si se quiere trabajar con nvm, hay que abrir una terminal en modo administrador.

allí ejecutando el comando **nvm** obtendremos la lista de comandos que se pueden utilizar, así como la versión, las opciones de instalación de versiones, etc.

nvm use 18.15.0 (por ejemplo)

nvm install latest (instala la última versión, pero suele estar disponible en versión de pruebas, por lo que no es recomendable)

Ver ejemplos de la carpeta 04.3 Node_pruebas01

La carpeta contiene sueltos los primeros ejemplos sencillos numerados desde 01ls.js

Un pequeños script que permite listar directorios con ls (en Windows, como si fuese un Unix, la cuestión es que por motivos de orden de archivos lo hemos denominado 01ls y 02ls2, por lo que habrá que utilizar esos nombres en vez de solo ls). El primero únicamente la carpeta donde se encuentra 01ls.js, el segundo

node 02ls2 (listará el contenido de la carpeta actual)

node 02ls2 .. (listará el contenido de la carpeta contenedora)

Recordad que los archivos node (aparte de este ejemplo, se ejecutan con la palabra node seguida del nombre del archivo y la extensión)

Node.js teoría 2

La utilización de util.promisify hace que podamos convertir una función que debe retornar algo, y que entonces hasta su respuesta se queda esperando), en una que retorna una promesa (promise). En realidad se trata de una función que utiliza el paradigma del error como primera respuesta de la llamada a retorno, y posteriormente una llamada a función que retorna una promesa. Dentro de las funciones asíncronas, las promises se administran muy bien.

Si ejecutamos node 03app.js (de la carpeta), veremos un primer servidor funcionando en local en el puerto 8124 y que saluda al mundo, como de costumbre

A diferencia de los otros scripts este no termina, ya que hay un event listener activo permanente. Si queremos bajar el mini servidor que se crea con este script hay que pulsar control c

NPM el administrador de paquetes de node

Es el repositorio de facto (no está establecido oficialmente). Resulta muy cómodo instalar dependencias desde allí, y es muy frecuente usarlo también con frameworks de front.

Con el código de node es posible trabajar sin preocuparse de que los navegadores no tengan actualizado el JS, ya que es lenguaje de servidor. Con la parte del front no tenemos tantas libertades. JavaScript está evolucionando más rápido de lo que los navegadores puedan actualizar sus intérpretes.

Un ejemplo son las **arrow functions**. Ejemplo:

```
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) ...; // haz algo con el error
  else ...; // haz algo con la data
});
```

Son mucho más que ahorrarnos escribir la palabra function. Las Arrow functions son más ligeras y fáciles de leer. La ligereza tiene el coste de cambiar el valor de this dentro de la arrow function. En las funciones this tiene un valor único dentro de la función. En una función de flecha, tiene el mismo valor que el ámbito que contiene la función de flecha. Esto significa que, cuando usamos una función arrow, no tenemos preocuparnos para this a la función de devolución de llamada porque es el mismo en ambos niveles del código.

Promise es una clase de la librería de node (un tipo de objeto), se utiliza cuando queremos aplazar algoritmos asíncronos. Recordad que el paradigma es siempre: el último argumento es la llamada a la función, y el primero es el indicador de error.

la clase Promise se crea de este modo:

```
function hazEsto(arg1, arg2) {
  return new Promise((resolve, reject) => {
    // ejecutamos código asíncrono
    if (errorIsDetected) return reject(errorObject);
    // cuando el proceso termina llama a esto
    resolve(result1, result2);
  });
}
```

pero en vez de pasar una llamada de retorno, es posible utilizar el .then (.entonces..)

```
hazEsto(arg1, arg2)
  .then(result => {
    // Esto sólo puede recibir un valor, por lo tanto para
    // recibir múltiples valores requiere un objeto o array
    return hazLoOtro(arg2, arg3);
  })
  .then((results) => {
    return hazOtyracosa(arg5);
  })
  .then(() => {
    // haz finalmente esto
  })
  .catch(err => {
    // aquí aterrizan los errores
  });
```

Esto funciona porque la clase Promise admite el encadenamiento si una función then devuelve un objeto Promise

Modulos de Node

Hay dos formatos de módulo que debemos considerar:

- El formato tradicional de Node.js basado en el estándar CommonJS se ha utilizado desde que se creó Node.js.
- Con ES2015/2016 se ha definido un nuevo formato, ES6 Modules, con una nueva palabra clave de importación. Los módulos ES6 son compatibles con todas las implementaciones de JavaScript.

Cada archivo de js que utilizamos es en sí, un módulo. Utilizamos

```
const fs = require('fs');
```

require busca un módulo con ese nombre y lo almacena en la const con el nombre que dimos. En el ejemplo fs es un módulo que viene por defecto (FileSystem)

Al mismo tiempo, el exports, de cada script especifica lo que queremos que se pueda recibir desde otros scripts o módulos

- Cualquier cosa (funciones u objetos) asignada como un campo de las exportaciones (como se conoce como module.exports) está disponible para otro código fuera del módulo
- Los objetos no asignados a exports no están disponibles para código fuera del módulo a menos que el módulo exporte esos objetos a través de otro mecanismo